

Ex. 1.9 — Three Papers from “OS History and Architecture”

From the “OS History and Architecture” group of the paper list (papers 1-1 through 1-11) I read **Paper 1-10** (Dijkstra, “THE”), **Paper 1-2** (Ritchie and Thompson, UNIX) and **Paper 1-3** (Engler, Kaashoek and O’Toole, Exokernel). I chose these three because they form a single argument carried across thirty years: structure by *layers*, then structure by *uniform abstraction*, then the proposal to *abolish* the abstraction altogether.

Paper 1-10 — Dijkstra, “The Structure of the “*THE*”-Multiprogramming System”

Summary. Dijkstra reports on a multiprogramming system built at Eindhoven for the Electrologica EL X8 — a machine with a $2.5\mu\text{s}$ core cycle, 27-bit words, 32K of core and a 512K-word drum — by six people of, on average, half-time availability. The goals are deliberately modest: shorter turnaround for short jobs, economical use of peripherals and automatic control of backing store. He is explicit that this is *not* a multi-access system; users share only the configuration and an ALGOL 60 procedure library.

Two structural ideas carry the design. The first decouples information from storage. “Segments” (units of information) are named independently of “pages” (units of memory, whether core or drum), and a segment variable in core records where a segment currently lives. A segment evicted to the drum therefore need not return to the drum page it came from, so the system simply picks the free page with minimum rotational latency and the drum allocation problem disappears entirely.

The second is the level hierarchy. The system is a society of sequential processes running at undefined speed ratios, coordinated only by explicit **P** and **V** operations on semaphores; the appendix defines these and separates semaphores used for mutual exclusion from *private* semaphores, on which only the owning process ever performs a P. The processes are stacked in a strict hierarchy: level 0 allocates the processor and absorbs the real-time clock; level 1 is the segment controller and the drum bookkeeping; level 2 is the message interpreter owning the console keyboard; level 3 buffers the peripherals; level 4 holds user programs; level 5 is the operator, “not implemented by us”. Each level implements an abstraction that makes the level below invisible — above 0 the number of processors is irrelevant, above 1 pages have lost their identity, above 2 every process appears to own a private console.

The claimed payoff is verification, not speed. Because a process can only generate tasks for processes at lower levels, circularity is excluded; harmonious cooperation is then proved in three stages, ending in the absence of circular waits — what Dijkstra calls “the Deadly Embrace”. Testing followed the same order, level 0 first and a new level added only once the one beneath it was exhausted. The reported outcome is that only trivial coding errors appeared, at a density of one per 500 instructions, each located within ten minutes.

Critical judgment. The case for layering here is purely structural, and the paper contains essentially no performance data — we are never told what the hierarchy *cost*, in memory, in context switches, or in the latency added by pushing every I/O request down through three levels. The most revealing sentence in the paper is the concession that at the time of writing testing was not yet complete, “but the resulting system is guaranteed to be flawless”. That is a claim about the rigour of a design being offered in place of evidence about an artifact, and the two are not the same thing. Worse, it is in tension with Dijkstra’s own report a paragraph earlier: what the proof covers is the synchronisation skeleton, whereas the errors he actually found were coding errors — exactly the class the proof cannot reach. He is transparent about both facts without ever conceding that the second undermines the first.

The strict ordering of levels also shows a stress crack that the paper itself records. Keeping the hierarchy acyclic required a special precaution: a segment requested from the drum must be pinned in core until the requesting process has actually touched it, since otherwise a finite task could generate an unbounded number of tasks for the segment controller and the system would thrash. That is a genuine cyclic dependency between levels 1 and 3 being patched to preserve the appearance of acyclicity, and it foreshadows why total ordering did not survive: in real systems the pager needs the disk driver which needs memory which needs the pager. Later designs kept layering as a discipline but abandoned it as a strict total order.

Finally, the scope does a great deal of quiet work in the argument. Six people, one known configuration, no multi-access, no user-written machine code, and no security model at all is a setting in which exhaustive testing is plausible in a way it is not elsewhere. Dijkstra anticipates precisely this objection — industry doubts the technique scales — and answers that “the larger the project, the more essential the structuring”. This may well be true, but as stated it is rhetoric rather than evidence, and it is the one claim in the paper with no support behind it. What did survive, and survived enormously, is the deeper thesis: that one should choose a structure *because* it makes correctness provable. That line runs directly from here to seL4.

Paper 1-2 — Ritchie and Thompson, “The UNIX Time-Sharing System”

Summary. This describes the third version of UNIX, on the PDP-11/40 and /45. The scale is part of the argument: a 16-bit machine with 144K bytes of core of which UNIX occupies 42K, hardware costing as little as 40,000 dollars, and less than two man-years spent on the main system software, with about 40 installations running since February 1971. The system had been rewritten in C during the summer of 1973; the C version is about a third larger than the assembler one, which the authors accept because it is far easier to understand and modify and brought multiprogramming and shared reentrant code with it.

The technical content is a small set of uniform abstractions. There are three kinds of file — ordinary, directory, and special. Ordinary files are unstructured strings of bytes: structure is the business of the programs that use a file, not of the system. Every device appears as a special file in `/dev`, which the authors argue has a threefold advantage — file and device I/O become as similar as possible, file and device names share one syntax, and devices fall under the same protection mechanism as files. Directories form a rooted tree with the conventional `.` and `..` entries, and all links to a file have equal status, because a directory entry holds nothing but a name and a pointer to an i-node. The i-node carries eight device addresses; small files use them directly and large files indirect through blocks of 256 addresses, which caps a file at $8 \cdot 256 \cdot 512 = 2^{20}$ bytes. `mount` grafts a removable volume’s hierarchy onto a leaf of the existing one.

Processes are equally spare. `fork` splits a process into two, `execute` overlays the image with a new program, and `wait/exit` join them again; `pipe` returns a descriptor for an interprocess channel that is read and written with the same `read` and `write` calls as a file. The Shell is not part of the kernel at all but an ordinary swappable user program that forks a process per command. Because the Shell — not the command — interprets `<` and `>`, no command contains any code for redirection; filters joined by `|` compose for the same reason. `init` creates one process per terminal line, and since a user’s password-file entry may name some other program in place of the Shell, the same mechanism confines the editing-only users and the users who may only play chess. Section 9 gives the operational picture: 72 users, 14 simultaneous, 4400 files, and per day 1800 commands, 4.3 CPU hours and 75 logins, at about 98% uptime with a longest unbroken run of roughly two weeks.

Critical judgment. The paper is unusually honest, and that honesty is simultaneously its

charm and its weakness as an evaluation. The entire performance argument is section 4.1: one assembly of a 7621-line program in 35.9 seconds, or 212 lines per second, split 63.5% assembler, 16.5% system overhead, 20% disk wait — immediately followed by the statement that the authors will not interpret these figures or compare them with any other system, and are “generally satisfied”. A design paper is entitled to that stance, but the consequence is that every efficiency claim in the paper rests on assertion. The famous “Perspective” remark, that UNIX succeeded largely because it was not designed to meet any predefined objectives, is charming and probably true, but it is unfalsifiable and useless as advice — one cannot plan to have no requirements.

The treatment of security is thin, and the mechanisms it introduces so casually became the classic problems. Protection is a user ID plus seven bits: read, write and execute for the owner and for everyone else — there is no group in this version — plus set-user-ID, plus a super-user exempt from all checking. Set-user-ID is presented purely as an elegance, available to any user on his own files without administrative intervention. The paper never asks what happens when such a program can be induced to act on its invoker’s behalf, and decades of privilege-escalation bugs live in that unexamined gap.

“Everything is a file” is also less total than the slogan it became, and the authors say so where it is inconvenient. Pipes are conceded to be “not a completely general mechanism” because a common ancestor must create them — the reason named pipes and then sockets had to be bolted on later, and the first place the uniform interface visibly leaks. The 1 MB file ceiling and the 14-character name limit are not principled choices but artifacts of the i-node layout. The accounting problem created by equal-status links is raised and then dropped with the admission that UNIX avoids the issue by charging no fees at all. There is essentially nothing on multiprocessing. And section 9 describes a research laboratory, not a production load — chess alone accounts for 5.3% of CPU time — so the reliability figures cannot support conclusions about UNIX under commercial pressure. What the paper does establish, and establishes decisively, is a negative result: the expense and bulk of contemporary operating systems were not intrinsic to the problem.

Paper 1-3 — Engler, Kaashoek and O’Toole, “Exokernel”

Summary. The target is the fixed abstraction itself. Any abstraction the kernel imposes — virtual memory, IPC — embodies a trade-off, between sparse and dense address spaces, or read-intensive and write-intensive access, and whichever way it is resolved some class of application pays. The authors cite measurements by Cao et al. showing that application-controlled file caching alone can cut running time by 45%. Their response is to separate protection from management: the kernel should securely multiplex the raw hardware and nothing more, while *library operating systems*, linked into the application’s own address space and untrusted, implement processes, virtual memory and IPC. Changing the operating system becomes relinking.

Three mechanisms make this safe. *Secure bindings* decouple authorisation from use: the kernel performs the expensive check once, when a binding is established, and afterwards enforces it cheaply. They are implemented by hardware support, by software caching — a large software TLB acting as a cache of frequently used bindings — or by downloading code into the kernel, of which packet filters are the paradigm case, application-supplied predicates that demultiplex packets without the owning application being scheduled. *Visible revocation* makes the library a participant rather than a victim: even the processor is explicitly revoked at the end of a time slice, so a library that knows its floating-point state is dead can decline to save it. When a library will not cooperate, the *abort protocol* breaks the bindings by force, records the seizure in a *repossession vector* and raises an exception, with a small guaranteed reserve of pages so the library can survive being reposessed.

The prototype is Aegis, which exports the processor, physical memory, TLB, exceptions, inter-

rupts and — via a packet filter built with dynamic code generation — the network; and ExOS, which implements processes, virtual memory, user-level exceptions, IPC and ARP/RARP, IP, UDP and NFS. Both run on MIPS DECstations and are compared against Ultrix 4.2. The headline numbers: exception dispatch in $1.5\mu\text{s}$ on a DECstation5000/125, roughly five times the best published figure and about two orders of magnitude faster than Ultrix; protected control transfer nearly seven times the best reported implementation; the dynamic packet filter classifying TCP/IP headers in $1.5\mu\text{s}$ against $35\mu\text{s}$ for MPF; and an application-level stride scheduler in under 100 lines of code. The authors conclude that the architecture is viable for high-performance extensible systems.

Critical judgment. The diagnosis is excellent and the evaluation cannot carry it. Nearly every number is a microbenchmark of a single primitive, obtained by repeating the operation many times and averaging, which — as the authors state themselves — means the measurements “do not consider cold start misses in the cache or TLB, and therefore represent a best case”. A system whose entire premise is that real applications are being held back by the kernel is evaluated without a real application. The one whole-program measurement, a 150×150 matrix multiplication, is dominated by the compiler and says almost nothing about the operating system underneath it.

The comparison is also structurally unfair in a way the paper concedes in a single sentence and then walks past: ExOS does “not offer the same level of functionality as Ultrix”, and “we do not expect these additions to cause large increases in our timing measurements”. That expectation is exactly the claim that needed evidence, because the absent functionality — swapping, and a real file system still described as under development — is precisely where a general-purpose system spends its money. The admission that the prototype “has no real users” should be read alongside the ten-to-100 $\times$ speedups: a factor measured against a system that is doing strictly more work is a much weaker result than the number suggests.

The security argument is asserted more than demonstrated. Secure bindings are supposed to preserve protection while removing management, but no adversary model is developed and no attack is attempted; the paper notes in passing that one must ensure a packet filter does not “lie” and accept packets destined for another application, and leaves the problem there. There is also a deep tension the authors do not resolve: exposing physical names and physical resources to untrusted software makes the interface hardware-specific by construction, which is the portability cost of the whole design, and their own dismissal of VM/370 — that virtualising the base machine is expensive — was overtaken within a decade by hardware virtualisation that made that cost largely vanish. History’s verdict is therefore split, and that is the fair judgment: the critique of fixed abstractions was right and has outlived the artifact, while the deployment argument was wrong. Library operating systems became practical roughly twenty years later, as unikernels and VM-hosted specialisation — which is to say the idea won on someone else’s mechanism.

The Three Together

Read in sequence these papers are one long argument about where structure should live. Dijkstra puts it in the kernel as a total order of levels, and buys provability with rigidity. Ritchie and Thompson put it in a handful of uniform abstractions — the byte stream, the i-node, the special file, the shell as an ordinary program — and buy extraordinary leverage at the price of leaks wherever the uniformity is false. Engler and his co-authors argue that any such fixed abstraction is a tax, and push structure out of the kernel entirely into replaceable libraries.

The pattern in their evidence is at least as instructive as the pattern in their designs. Dijkstra offers a proof and no measurements; Ritchie and Thompson offer one measurement they decline to interpret; the exokernel offers a great many measurements, all of primitives and none of a workload anyone actually ran. Each paper is strongest exactly where the previous one was weakest and

weakest in a new place — which is a reasonable summary of how the field made progress.

References

- [1-10] E. W. Dijkstra. “The Structure of the “*THE*”-Multiprogramming System.” *Communications of the ACM*, vol. 11, no. 5, May 1968, pp. 341–346. (Presented at the ACM Symposium on Operating System Principles, Gatlinburg, Tennessee, October 1967; also circulated as EWD196.)
- [1-2] D. M. Ritchie and K. Thompson. “The UNIX Time-Sharing System.” *Communications of the ACM*, vol. 17, no. 7, July 1974, pp. 365–375. (Revised version of a paper presented at the Fourth ACM Symposium on Operating Systems Principles, Yorktown Heights, New York, October 1973.)
- [1-3] D. R. Engler, M. F. Kaashoek and J. O’Toole Jr. “Exokernel: An Operating System Architecture for Application-Level Resource Management.” *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP ’95)*, 1995, pp. 251–266.